

Full Page Handwriting Recognition via Image to Sequence Extraction

Sumeet S. Singh¹ (✉)^[0000-0002-5323-9678] and Sergey Karayev¹

¹ Turnitin, 2101 Webster St #1800, Oakland, CA 94612, USA
 ssingh@turnitin.com / sumeet@singhonline.info, sergeykarayev@gmail.com

Abstract. We present a Neural Network based Handwritten Text Recognition (HTR) model architecture that can be trained to recognize full pages of handwritten or printed text without image segmentation. Being based on Image to Sequence architecture, it can extract text present in an image and then sequence it correctly without imposing any constraints regarding orientation, layout and size of text and non-text. Further, it can also be trained to generate auxiliary markup related to formatting, layout and content. We use character level vocabulary, thereby enabling language and terminology of any subject. The model achieves a new state-of-art in paragraph level recognition on the IAM dataset. When evaluated on scans of real world handwritten free form test answers - beset with curved and slanted lines, drawings, tables, math, chemistry and other symbols - it performs better than all commercially available HTR cloud APIs. It is deployed in production as part of a commercial web application.

1 Overview

With the advancement of Deep Neural Networks, the field of HTR has progressed by leaps and bounds. Neural Networks have enabled algorithm developers to increasingly rely on features and algorithms learned from data rather than hand crafted ones. This makes it easier than ever before to create HTR models for new datasets and languages. That said, HTR models still embody domain-specific inductive biases, assumptions and heuristics in their architectures regarding the layout, size and orientation of text. We aim to address some of these limitations in this work.

1.1 Challenges of Real World Full Page HTR

Our target data - *Free Form Answers* dataset - was derived from scans of test paper submissions of STEM subjects, from school level all the way up to post-graduate courses. It consists of images containing possibly multiple blurbs of handwritten text, math equations, tables, drawings, diagrams, side-notes, scratched out text and text inserted using an arrow / circumflex and other artifacts, all put together with no reliable layout (Figures 1 and 2). Length of transcription ranges from 0 to 1100 characters averaging around 160.

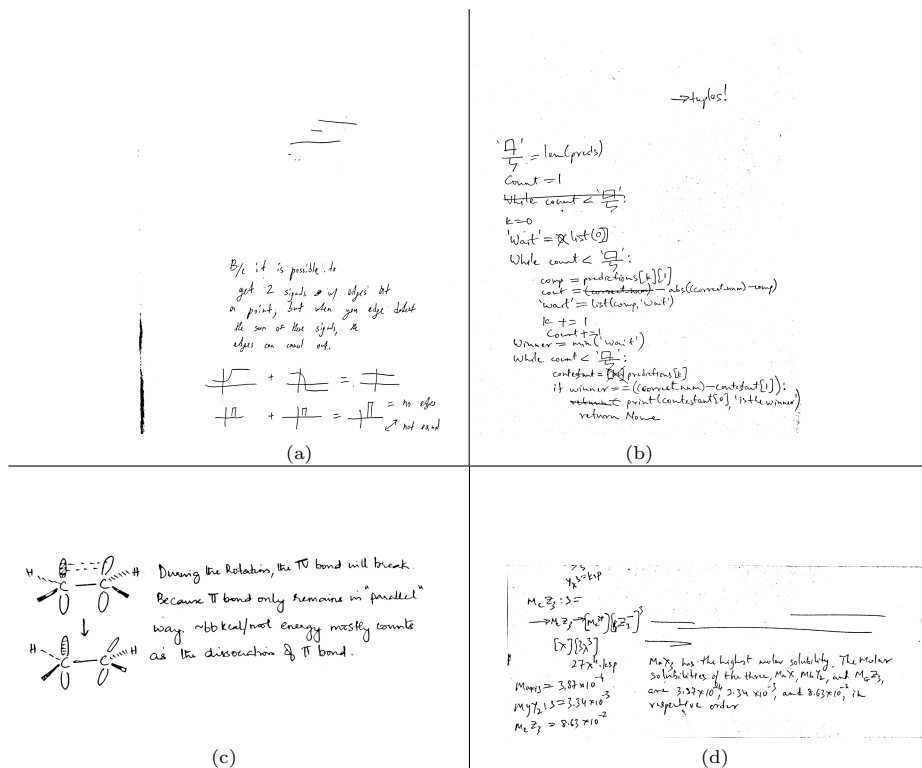


Fig. 1: Samples from our Free Form Answers dataset. (a) Full page text with drawing. (b) Full page computer source code. (c) Diagrams and text with embedded math. (d) Math and text regions, embedded math and stray artifacts.

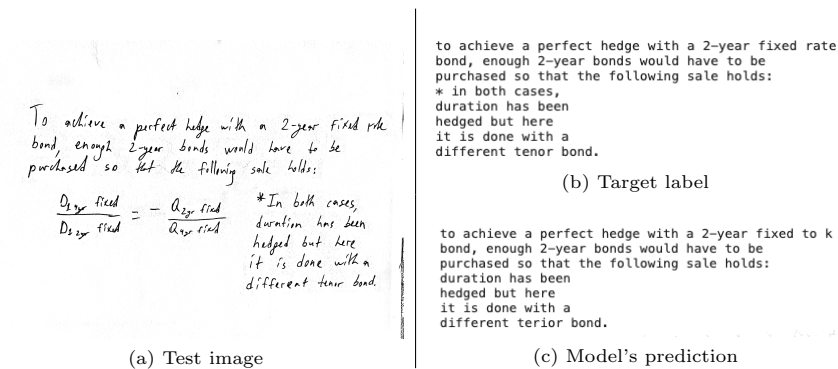


Fig. 2: Test image with two text and one math regions. The model successfully skips the math region and transcribes the two text regions, with some mistakes.

Problem Framework In order to get a handle on this seemingly random structure, we define each independent blurb as a *region* and view each image as being composed of one or more regions. The regions are further classified as text, math, drawing, tables and deleted (scratched) text. Non textual regions and deleted text (hereafter *untranscribed regions*) are optionally demarcated with special *auxiliary tags* but left untranscribed otherwise. Text regions range from a few characters to multiple paragraphs and are possibly interspersed or embedded with untranscribed regions. Text was transcribed in the order it was meant to be read¹ and line ends, empty lines, spaces and indentations are also faithfully transcribed. These can be programmatically removed later if so desired. Since the regions can be randomly situated on the image, we define a predictable region sequencing order *the natural reading order* as the order in which somebody would read the answer aloud. This order is implicitly captured in the transcription and the model must learn to reproduce it.

With the above framework in mind, the problem becomes: 1) Implicitly identify and classify regions on the image 2) Extract text from each text region and emit it in natural reading order 3) Ignore / skip unrecognized artifacts and 4) Produce auxiliary markup. We call the above formulation *extractive*, since it seeks to extract the identified and desirable content but ignore the unknown and undesirable. This formulation is generic enough to cover simple to complex scenarios but also teaches the model to skip over artifacts that were not encountered in the dataset.

1.2 Limitations of Existing Architectures

Most state-of-the-art HTR models rely on a prior image segmentation method to cleanly isolate pieces of text (words, lines or paragraphs). There are several problems with this approach.

First, image segmentation in such models is a separate system, usually based on hand-crafted features and heuristics which do not hold up when the data changes significantly. Some methods go even further and correct segmented text for slant, curve, and height, using the same problematic features and heuristics.

Second, and more importantly, clean segmentation of units of text is not even possible in many cases of real world handwriting. For instance, observe that lines are curved or interspersed with non-textual symbols and artifacts in Figures 1 and 2. Further discussion of such limitations can be found in the literature [3, 4].

Third, stitching a complete transcription from the individually transcribed text regions introduces yet another system, with its own potentials for errors, and brittleness to changing the data distribution (for example, right-to-left languages would require a different stitching system).

Lastly, formatting and indentation tends to get lost in this three-step process because text segments are usually concatenated using space or newline as a separator. This would be unacceptable when transcribing Python language source code, for instance.

¹This becomes relevant when text is not horizontal or when inserted using a circumflex or arrow.

In our end-to-end model all of the above steps are implicit and learned from data. Adapting the model to a new dataset or adding new capabilities is just a matter of retraining or fine-tuning with different labels or different data.

1.3 Model Overview

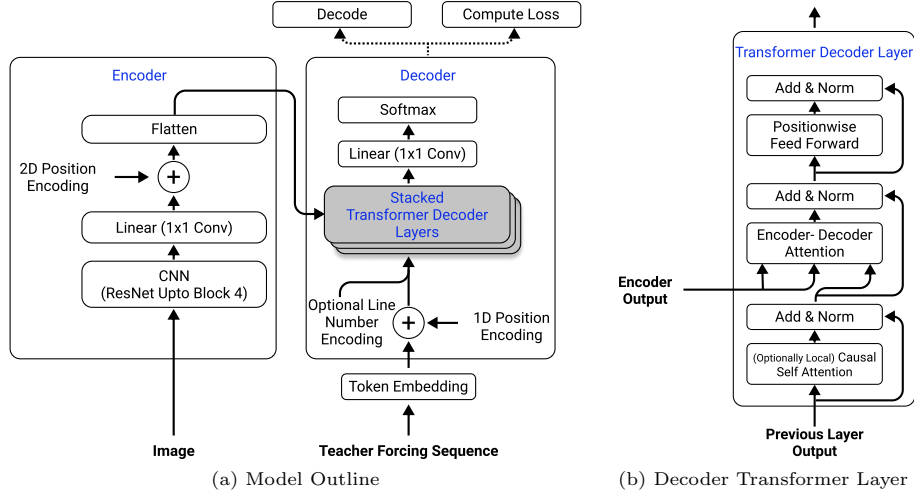


Fig. 3: Model Schematics. Left: CNN Encoder and Transformer Decoder. Right: The Transformer Layer is identical to [28] except that Self Attention maybe optionally Local. Teacher Forcing Sequence refers to the ground-truth shifted right during training or the predicted tokens when predicting.

Image to Sequence Architecture Our model (Figure 3) belongs to the lineage of attention-based sequence-to-sequence and tensor-to-tensor architectures [29, 33]. It consists of a ResNet [11] *encoder* and a Transformer [28] *decoder*. It has a *Image to Sequence architecture* i.e., it learns to map an image to a sequence of tokens. The model signals end of output sequence via a special $\langle \text{EOS} \rangle$ token, so there is no limit on sequence length².

Formatting and Auxiliary Markup We trained the model to extract text from text regions only, and to either skip the the untranscribed regions or produce special markup tags ($\langle \text{END-OF-REGION} \rangle$, $\langle \text{MATH} \rangle$, $\langle \text{DELETED-TEXT} \rangle$, $\langle \text{TABLE} \rangle$ AND $\langle \text{DRAWING} \rangle$) for them. This requires the model to identify the untranscribed regions, which are almost always *unique per sample*. It generalizes

²Except a limit set at prediction to prevent an endless loop.

reasonably well in either of these tasks. Similarly, because whitespace is preserved in our ground truth transcription, the model learns to faithfully replicate multiple empty lines and indentation spaces, which is important for computer source code for example.

Token Vocabulary In order to cater to a variety of subject matter terminology and proper nouns, we chose to use character level token vocabularies in our base configuration. That said, we experimented with different types of vocabularies, for example character vs hybrid word/character, or lowercase vs mixed case. Model performance did not differ significantly between these variations. Therefore, we are confident that more complex transcription tasks – such as transcribing math [6, 26] and complex layouts such as tables – should be possible given appropriately transcribed training data and a character-level vocabulary.

Layout Agnostic Architecture Since there are no assumptions regarding the layout of text, the model works equally well on single-column vs two-column text, and easily learns to produce a column separator tag (Table 2). The multistage HTR models described in subsection 1.2 on the other hand, would concatenate adjacent lines from the two columns, unless a separate code module was added to recognize columns and modifications made to the rest of the code.

Performance The model outperforms the best commercially available API on our proprietary Free Form Answers dataset (Table 2) and state of art full page recognition [4] and full paragraph recognition [3] models on the academic IAM dataset (Table 1).

2 Related Work

The first work to use Neural Networks for offline HTR was by Lecun et al. [15] which recognized single digits. The next evolution was by Graves and Schmidhuber [10] who used an MDLSTM model and CTC loss and decoding [9] to recognize Tunisian town names in Arabic. Their model was designed to recognize a single line of text, oriented horizontally. [22] refined the MDLSTM+CTC model by adding dropout, but it remained a single-line recognition model dependent on prior line segmentation and normalization. Voigtlaender et al. [30] further increased performance by adding deslanting preprocessing (which is entirely dataset specific) and using a bigger network.

Bluche et al. [4] developed a model with the same vision as ours: full-page recognition with no layout or size assumptions. However, they subsequently abandoned this approach citing prohibitive memory requirements, unavailability of GPU acceleration for training of MDLSTM and intractable inference time. Follow-up work by the same authors [2, 3] saw a come-back to the encoder-only + CTC approach but with a scaled-back MDLSTM attention model that could isolate individual lines of text, in effect performing automatic line segmentation

and enabling the model to recognize paragraphs. While far superior to single-line recognition models, this approach still hard-codes the assumption that lines of text stretch horizontally from left to right, and fill the entire image. Therefore, it can't handle arbitrary text layouts and instead, relies on paragraph segmentation. This approach also does not output a variable-length sequence rather a fixed number of lines T (some of which may be empty), T being baked into the model during training. Finally, since the predicted lines are joined together using a fixed separator it is unlikely that the model can faithfully reproduce empty lines and indentation.

Further on, embracing the trend towards more parallel architectures, Bluche and Messina [2] and Puigcerver [23] replaced the MDLSTM encoder with CNN, but continued to rely on CTC.

Another recent trend in deep learning has been the move away from encoder-only towards encoder-decoder sequence-to-sequence architectures. These architectures decouple the sequence length of the output from the input thereby eliminating the need for CTC loss and decoding. Cross-entropy loss and greedy / beam-search decoding which are less compute-intensive are employed instead. Ly et al. [16] applied such architecture to recognizing very short passages of Japanese text (a handful of short vertical lines). Similarly Wang et al. [31] applied a bespoke sequence-to-sequence architecture for recognizing single lines.

It is well established now that Transformers can completely replace LSTMs in function and are more parameter efficient, accurate and enable longer sequences. Embracing this trend Kang et al. [13] published the most 'modern' architecture to date employing CNN and Transformers somewhat like ours. However since it collapses the vertical dimension of the image feature map, this model is designed to recognize single lines only. It also employs a transformer encoder thereby making it larger than our model. To our knowledge, ours is the only work other than [4] that attempts full page handwriting recognition.

3 Model Architecture

Our Neural Network architecture is shown in Figure 3. It is an encoder-decoder architecture, using ResNet [11] for encoding the image, and Transformer [28] for decoding the encoded representation into text. We refer you to [26–28, 33] for a background on neural image-to-sequence and sequence-to-sequence models. This section will fill in the remaining details necessary to reproduce our model architecture.

We use the term *base configuration* to refer to our most frequently used model configuration, and all configuration parameters we list hereafter describe this configuration unless stated otherwise.

Encoder The encoder uses a CNN to extract a 2D feature-map from the input image. It uses the ResNet architecture without its last two layers: the average-pool and linear projection. The feature-map is then projected to match the

Transformer’s hidden-size d_{model} , then a 2D positional encoding added and finally flattened into a 1D sequence. 2D positional encoding is a fixed sinusoidal encoding as in [28], but using the first $d_{model}/2$ channels to encode the Y coordinate and the rest to encode the X coordinate (Equation 1) (similar to [20]). Output $\mathbf{I}$ of the *Flatten* layer is made available to all Transformer decoder layers, as is standard.

$$\begin{aligned}
 PE(y, 2i) &= \sin(y/10000^{2i/d_{model}}) \\
 PE(y, 2i + 1) &= \cos(y/10000^{2i/d_{model}}) \\
 PE(x, d_{model}/2 + 2i) &= \sin(x/10000^{2i/d_{model}}) \\
 PE(x, d_{model}/2 + 2i + 1) &= \cos(x/10000^{2i/d_{model}}) \\
 i &\in [0, d_{model}/4)
 \end{aligned} \tag{1}$$

Decoder The decoder is a Transformer stack with non-causal attention to the encoder output (its layers can attend to the encoder’s entire output) and causal self-attention (it can only attend to past positions of its text input). As is standard, training is done with *teacher forcing*, which means that the ground truth text input is shifted one off from the output.

In total, the base configuration has 27.8 million parameters (6.3M decoder, 21.4M ResNet).

The input vectors are enhanced with 1D position encoding, as is standard. Additionally, we concatenate a *line number encoding* (*lne*) - the scaled text line number (l) that the token lies on - to it (Figure 3a). Assuming a maximum of 100 text lines, $l \in [1, 100]$ and $lne = l/100$. We added *lne* to the model in order to address line level errors i.e., missing or duplicated lines but it was applied only in Table 1. We haven’t yet officially concluded on its impact on model performance and mention it here only for completeness sake.

In order to improve memory and computation requirements of our model, we implemented a localized form of causal self-attention by limiting the attention span to 50 (configurable) past positions. This is similar to Sliding Window Attention of [1]) or a 1D version of Local Self Attention of [20]). We hypothesized that a look back of 50 characters should be enough to satisfy the language modeling needs of our task, while the limited attention span should help training converge faster, both assumptions being validated by experiment. Final model performance however, was not impacted by it. That said, a thorough ablation study was not performed. Practically though, it allowed us to use larger mini-batches by about 12%.

Objective Function For each step t the model outputs the token probability distribution $\mathbf{p}_t$ over the vocabulary set $\{1, \dots, V\}$ (Equation 2). This distribution is conditioned upon the tokens generated thus far $\mathbf{y}_{<t}$ and $\mathbf{I}$ (Equation 3).

Probability of the entire token sequence is therefore given by Equation 4.

$$\mathbf{p}_t : \{1, \dots, V\} \rightarrow [0, 1] \quad ; \mathbf{Y}_t \sim \mathbf{p}_t \quad (2)$$

$$\mathbf{p}_t(\mathbf{y}_t) := \mathbb{P}(\mathbf{Y}_t = \mathbf{y}_t | \mathbf{y}_{<t}, \mathbf{I}) \quad (3)$$

$$\mathbb{P}(\mathbf{y} | \mathbf{I}) = \prod_{t=1}^{\tau} \mathbf{p}_t(\mathbf{y}_t) \quad (4)$$

As is typical with sequence generators, the training objective here is to maximize probability of the target sequence $\mathbf{y}^{GT}$. We use the standard per-word cross-entropy objective (Equation 5), modified slightly for the mini-batch (Equation 6). We did not use any regularization objective, relying instead on dropout, data-augmentations and synthetic data to provide regularization.

$$\mathcal{L}_{seq} = -\frac{1}{\tau} \sum_t \ln(\mathbf{p}_t(\mathbf{y}_t^{GT})) \quad ; \tau \equiv \text{sequence length} \quad (5)$$

$$\mathcal{L}_{batch} = -\frac{1}{n} \sum_{batch} \sum_t \ln(\mathbf{p}_t(\mathbf{y}_t^{GT})) \quad ; n \equiv \# \text{ of tokens in batch} \quad (6)$$

The final Linear layer of the decoder (Figure. 3a) is a 1x1 convolution function that produces logits which are then normalized by softmax to produce $\mathbf{p}_t$.

Combination of Vision and NLP One of the strengths of our architecture is in the combination of Vision and Language models. CNNs such as ResNet are considered best for processing image data. And Transformers are considered best for Language Modeling (LM) and Natural Language Understanding (NLU) tasks [7, 24, 25], possessing properties that are very useful in dealing with noisy and incomplete text that often occurs in real handwriting. Having both the visual feature map and a language model, the model can do a much better job than one relying on visual features alone.

Inference We use simple greedy decoding, which picks the highest probability token at each step. Beam search decoding [8] did not yield any accuracy improvement indicating that the model is quite opinionated / confident.

4 Training Configuration and Procedure

The base configuration uses grayscale images scaled down to 140-150 dots per inch. Higher resolutions yielded slightly better accuracy at the cost of compute and memory. We use the 34-layer configuration of ResNet, but have also successfully trained the 18-layer and 50-layer configurations; larger models tending to do better in general as expected.

The following is the base configuration of the Transformer stack:

- N (number of layers) = 6

- $d_{model} = 260$
- h (number of heads) = 4
- d_{ff} (inner-layer of positionwise feed-forward network) = 1024
- Activation function inside feed-forward layer = GELU [12]
- dropout = 0.5

The model was implemented in PyTorch [21], and training was carried out using 8 NVIDIA 2080Ti GPUs. For full page datasets a mini-batch size of 56 combined with a gradient accumulation factor of 2 was used, yielding an effective batch-size of 112. Single-line datasets had batch sizes as high as 200, but were adjusted downwards when using higher angles of image rotation. ADAM optimizer [14] was employed with a fixed learning rate (α) of 0.0002, $\beta_1 = 0.9$ and $\beta_2 = 0.999$.

While all images in a batch must have the same size; we also set all batches to have the same image size, padding smaller images as needed. This helps during training because any impending GPU OOM errors surface quickly at the beginning of the run. It also makes the validation / test results agnostic of the batching scheme since the images will always be the same size regardless of how they are grouped. Smaller images within a batch are centered during validation and testing. Padding color can be either the max of 4 corner pixels or simply 0 (black), the choice having no impact on model accuracy.

The base configuration vocabulary consists of all lowercase ASCII printable characters, including space and newline.

We observed that model performance increases with increased layer sizes and also image resolution. It also tends to improve monotonically with training time. Therefore, we trained our models for as long as possible the longest being 11 days on full pages (~47M samples) and 8 days (~102M samples) on single lines. Typical training length though is roughly 24M total training samples. Model state is saved periodically during training. At the end of training, the checkpoint that performs best on validation is selected for final testing.

5 Data

Data Sources The following is a comprehensive set of all data sources, although each experiment used only a subset:

- IAM: The IAM dataset [17] with the RWTH Aachen University split [19], which is a widely used benchmark for handwriting recognition.
- WIKITEXT: The WikiText-103 [18] dataset was used to generate synthetic images (explained later).
- FREE FORM ANSWERS: Our proprietary dataset with about 13K samples described in subsection 1.1.
- ANSWERS2: Another proprietary dataset of about 16K segmented words and lines extracted from handwritten test answers. However since we train full pages, we stitched back neighboring words and lines using heuristics.

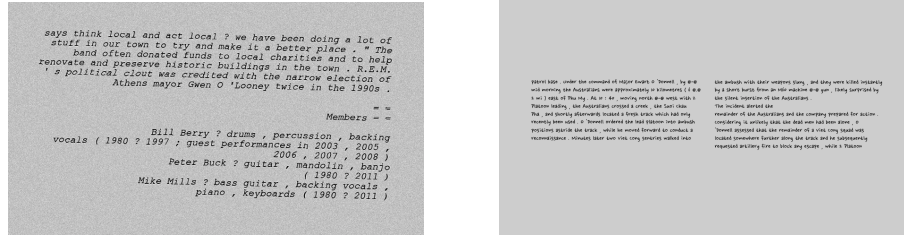


Fig. 4: Synthetic samples generated from WikiText-103 Dataset [18]. For the two column sample (b), the model – as per training – predicts the left column first, then a column separator tag `<col>` and then the right column.

- NAMES: Yet another proprietary dataset of about 22K handwritten names and identifiers.

Proprietary datasets were only used for results reported in Table 2.

Synthetic Data Augmentation Since the IAM data set has word-level segmentation, we generate synthetic IAM samples by stitching together images of random spans of words or lines. This made it possible to significantly augment the IAM paragraph dataset beyond the mere 747 training forms available in the RWTH Aachen split. Without this augmentation the model would not generalize at all.

We also generate synthetic images on the fly from the WikiText data by picking random text spans and rendering them into single-column and/or two-column layout, using 34 fonts in multiple sizes for a total of 114 combinations (Figure 4). The WikiText data is over 530 million characters long from which over 583 billion unique strings of lengths ranging from 1 to 1100 may be created. Multiplying this with 114 (fonts) yields an epoch size of 66.46 trillion which we would never get through in any of our runs. The dataset thus provides us with a continuous stream of unique samples which builds the implicit language model. Furthermore this trick can be used to ‘teach’ the model new language and terminology by using an appropriate seed text file. Addition of this dataset reduces the error rate on IAM paragraphs by only about 0.4% (Table 1.) on the IAM dataset but significantly improves the validation loss - about 30%.

Additionally, we generate empty images of varying backgrounds on the fly. Without this data, the model generates text even on blank images – which provides evidence of an underlying language model working a little overzealously.

Image Augmentation The following transformations were applied to individual training images: 1) Image scale, 2) rotation, 3) brightness, 4) background color of synthetic images, 5) contrast, 6) perspective and 7) Gaussian noise. At the batch level the images were randomly placed within the larger batch image size during training but centered during validation and testing.

Data Sampling We found that the model generalized best when prior biases were eliminated from the training data. Therefore the datasets are sampled on the fly for every mini-batch with a configurable distribution. We also do not group images by size or sequence length rather we sample them uniformly. Further, parameters for synthetic data generation (e.g. the text span to render or the font to render it in) and image augmentation (e.g. scale of image, angle of rotation, background color) are also sampled uniformly.

In spite of this sampling scheme, one bias does enter into the training data: padding around text. This is so because most images of a batch are smaller than the batch image size and therefore get padded. This causes the model to perform poorly in inference unless the image was sufficient padding. The optimal padding amount becomes a new hyperparameter that requires tuning after training. We circumvented this problem by padding all images to the same (large) size both during training and inference. This scheme though has the downside of consuming the highest amount of encoder compute regardless of the original image size. Therefore the first scheme is preferable for production.

6 Results

Table 1 shows character error rates (Levenshtein edit distance divided by length of ground truth) of paragraph and single line level tests on the IAM dataset. FPHR refers to our model trained with only the IAM dataset whereas FPHR+Aug refers to our model trained with the IAM plus WikiText based synthetic dataset³. FPHR trained at ~145 DPI outperforms previous state of art in the paragraph level test⁴ and FPHR+Aug improves error rate by a further 0.4%. When trained on single lines only, performance is similar to full page but short of the specialized state-of-the-art single-line models. This is because the model does equally well on different text lengths and number of lines provided they were uniformly distributed during training i.e., performance tracks the training data distribution. Notice that our corpus level CERs are lower than the averages indicating better performance on longer text. This is not a characteristic of the model, rather that of the training data which had fewer short sequences. We believe that the observed performance (6+% test and 4+% validation CER) is the upper limit of this system and task configuration i.e., the model architecture, its size, the dataset and its split. Increasing the model size and / or image resolution does improve performance slightly (albeit by less than 1%) as does increasing the amount of training data.

When trained on all datasets and evaluated on Free Form Answers, our model gets a error rate of 7.6% vs the best available cloud API's 14.4% (Table 2).

³We view synthetic WikiText based data as an augmentation method since it does not rely on proprietary data or method.

⁴Results from [2] are not included because it was trained on a lot more than IAM data and 30% of it was proprietary.

Favoring the cloud models, we removed auxiliary markup, line indentations and lower cased all predicted and ground truth text for this comparison⁵.

The model has no trouble transcribing two column layout; its performance on two-column and single-column data are comparable (Table 2) providing evidence of its adaptability to different text layouts. The Cloud API on the other hand does well on single-column data, but falters with two-column text precisely because it concatenates adjacent lines across columns.

Inferencing takes an average of 4.6 seconds on a single CPU thread for a set of images averaging 2500x2200 pixels, 456 chars and 11.65 lines without model compression i.e., model pruning, distillation or quantization.

Table 1: Comparison on the IAM dataset with and without closed lexicon decoding (LM). Figures in brackets are corpus level scores. [★]Model requires paragraph segmentation. [‡]FPHR trained with single lines only. SLS = Shredding Line Segmentation.

Test Type	Model	Mean Test CER w/o LM	Mean Test CER w/ LM
Paragraph Level	Bluche et al. [4] (150 dpi)	16.2%	
	FPHR (~145 dpi)	6.7 (6.5)%	
	FPHR+Aug (~145 dpi)	6.3 (6.1)%	
Paragraph Level [★]	Bluche [3] (150 dpi)	10.1%	6.5%
	Bluche [3] (300 dpi)	7.9%	5.5 %
	SLS + MDLSTM + CTC [4] (150 dpi)	11.1%	
	SLS + MDLSTM + CTC [4] (300 dpi)	7.5%	
Single Line	Puigcerver [23]	5.8%	4.4 %
	Wigington et al. [32]	6.4%	
	Wang et al. [31]	6.4%	
	Kang et al. [13]	4.7 %	
	FPHR+Aug [‡] (~145 dpi)	6.5 (5.9)%	

7 Conclusion

We have presented a “modern” neural network architecture that can be trained to perform full page handwriting recognition without image segmentation, delivers state of art accuracy and which is also small and fast enough to be deployed into commercial production. It adapts reasonably to different vocabularies, text layouts and auxiliary tasks and is therefore fit to serve a variety of handwriting

⁵We evaluated Microsoft, Google and Mathpix cloud APIs. Microsoft performed the best and its results are reported here. This is not intended to be a comparison of models, rather a practical data point that can be used to make build-vs-buy decisions.

Table 2: Character Error Rates on Free Form Answers and multi column synthetic datasets. *FPHR trained on one and two col. WikiText synthetic data.

Test Data set	Best Cloud API	FPHR
Free Form Answers	14.4%	7.6%
Wikitext (1 column)	1.4%	0.008%*
Wikitext (2 column)	57%	0.012%*

and printed text recognition scenarios. The model is also quite easy to replicate using open source ‘off the shelf’ modules making it all the more compelling. Although the overall Full Page HTR problem is not solved yet, we believe it takes us one step forward from [4] and [3].

7.1 Limitations and Future Work

Although the presented framework encompasses multiple tasks, available datasets are usually heavily biased towards one or two thereby masking the model’s performance on outlier tasks. For e.g., there’s usually only one transcribed text region per sample in the Free Form dataset which makes the model tend to transcribe only one (main) text region while skipping others. On the other hand when the dataset is balanced e.g., with one and two column synthetic text, it performs well on both layouts. That said, this aspect needs to be explored more thoroughly and hopefully with standardized datasets and tasks so that the research community can iterate over it.

Second, we have only trained with text up to 1100 characters long and averaging 360 characters. Should there be a need to transcribe longer lengths of text say 10K characters, then some more work becomes necessary in order to deal with longer sequence lengths. Possible solutions include the use of multicharacter vocabularies and sparse Transformers such as [1, 5].

Other desirable improvements to the model include 1) reducing its sensitivity to image padding. 2) Reducing the encoder’s size, which currently stands at almost 22 million parameters. 3) Separating vision models for significantly different visual data (e.g., Synthetic v/s Free Response Answers) so that they may both contribute to the language model but not interfere with each other’s vision models.

We believe that the Full Page HTR problem cannot be considered “solved” until the error rate has been brought down to less than 1%. Therefore, more work remains for the community.

Acknowledgements We would like to thank Saurabh Bipin Chandra for implementing the fast inference path ($O(N^2)$) of the Transformer decoder, which was lacking in PyTorch.

Bibliography

1. Beltagy, I., Peters, M.E., and Cohan, A. Longformer: The long-document transformer, 2020.
2. Bluche, T. and Messina, R. Gated convolutional recurrent neural networks for multilingual handwriting recognition. In *2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR)*, volume 01, pages 646–651, 2017.
3. Bluche, T. Joint line segmentation and transcription for end-to-end handwritten paragraph recognition. *ArXiv*, abs/1604.08352, 2016.
4. Bluche, T., Louradour, J., and Messina, R.O. Scan, attend and read: End-to-end handwritten paragraph recognition with MDLSTM attention. *CoRR*, abs/1604.03286, 2016.
5. Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q.V., and Salakhutdinov, R. Transformer-xl: Attentive language models beyond a fixed-length context, 2019.
6. Deng, Y., Kanervisto, A., Ling, J., and Rush, A.M. Image-to-markup generation with coarse-to-fine attention. In *ICML*, 2017.
7. Devlin, J., Chang, M.W., Lee, K., and Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
8. Graves, A. Supervised sequence labelling with recurrent neural networks. In *Studies in Computational Intelligence*, 2008.
9. Graves, A., Fernández, S., Gomez, F., and Schmidhuber, J. Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In *ICML '06*, 2006.
10. Graves, A. and Schmidhuber, J. Offline handwriting recognition with multidimensional recurrent neural networks. In *NIPS*, 2008.
11. He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. *CoRR*, abs/1512.03385, 2015.
12. Hendrycks, D. and Gimpel, K. Bridging nonlinearities and stochastic regularizers with gaussian error linear units. *CoRR*, abs/1606.08415, 2016. URL <http://arxiv.org/abs/1606.08415>.
13. Kang, L., Riba, P., Rusiñol, M., Fornés, A., and Villegas, M. Pay attention to what you read: Non-recurrent handwritten text-line recognition, 2020.
14. Kingma, D.P. and Ba, J. Adam: A method for stochastic optimization, 2017.
15. Lecun, Y., Bottou, L., Bengio, Y., and Haffner, P. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
16. Ly, N.T., Nguyen, C.T., and Nakagawa, M. An attention-based end-to-end model for multiple text lines recognition in japanese historical documents. In *2019 International Conference on Document Analysis and Recognition (ICDAR)*, pages 629–634, 2019. <https://doi.org/10.1109/ICDAR.2019.00106>.
17. Marti, U.V. and Bunke, H. The iam-database: An english sentence database for offline handwriting recognition. *International*

- Journal on Document Analysis and Recognition*, 5:39–46, 11 2002. <https://doi.org/10.1007/s100320200071>.
18. Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. *CoRR*, abs/1609.07843, 2016.
 19. Open SLR. Aachen data splits (train, test, val) for the iam dataset. <https://www.openslr.org/56/>. Identifier: SLR56.
 20. Parmar, N., Vaswani, A., Uszkoreit, J., Łukasz Kaiser, Shazeer, N., Ku, A., and Tran, D. Image transformer, 2018.
 21. Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. Pytorch: An imperative style, high-performance deep learning library. In Wallach, H., Larochelle, H., Beygelzimer, A., d'Alché-Buc, F., Fox, E., and Garnett, R., editors, *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019. URL <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>.
 22. Pham, V., Kermorvant, C., and Louradour, J. Dropout improves recurrent neural networks for handwriting recognition. *CoRR*, abs/1312.4569, 2013. URL <http://arxiv.org/abs/1312.4569>.
 23. Puigcerver, J. Are multidimensional recurrent layers really necessary for handwritten text recognition? In *2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR)*, volume 01, pages 67–72, 2017.
 24. Radford, A. Improving language understanding by generative pre-training. 2018.
 25. Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P.J. Exploring the limits of transfer learning with a unified text-to-text transformer, 2020.
 26. Singh, S.S. Teaching machines to code: Neural markup generation with visual attention. *CoRR*, abs/1802.05415, 2018.
 27. Sutskever, I., Vinyals, O., and Le, Q.V. Sequence to sequence learning with neural networks. *CoRR*, abs/1409.3215, 2014. URL <http://arxiv.org/abs/1409.3215>.
 28. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., and Polosukhin, I. Attention is all you need. *CoRR*, abs/1706.03762, 2017.
 29. Vaswani, A., Bengio, S., Brevdo, E., Chollet, F., Gomez, A.N., Gouws, S., Jones, L., Kaiser, L., Kalchbrenner, N., Parmar, N., Sepassi, R., Shazeer, N., and Uszkoreit, J. Tensor2tensor for neural machine translation. *CoRR*, abs/1803.07416, 2018.
 30. Voigtlaender, P., Doetsch, P., and Ney, H. Handwriting recognition with large multidimensional long short-term memory recurrent neural networks. In *2016 15th International Conference on Frontiers in Handwriting Recognition (ICFHR)*, pages 228–233, 2016.

31. Wang, T., Zhu, Y., Jin, L., Luo, C., Chen, X., Wu, Y., Wang, Q., and Cai, M. Decoupled attention network for text recognition, 2019.
32. Wigington, C., Tensmeyer, C., Davis, B., Barrett, W., Price, B., and Cohen, S. Start, follow, read: End-to-end full-page handwriting recognition. In *Proceedings of the European Conference on Computer Vision (ECCV)*, September 2018.
33. Xu, K., Ba, J., Kiros, J.R., Cho, K., Courville, A.C., Salakhutdinov, R., Zemel, R.S., and Bengio, Y. Show, attend and tell: Neural image caption generation with visual attention. In *ICML*, 2015.